CO2 – AT1 – Database Design and Connectivity

Sample Questions

1. Student Database Design and Connectivity (10 Marks)

A college wants to automate the management of student records. Design an SQLite database containing a **Student** table with appropriate fields and a primary key. Write a Python program to connect to the SQLite database, create the table if it does not exist, insert at least five student records, and display all the records stored in the table.

CODE:

app.py - C:/Users/hp/Downloads/student/app.py (3.13.9)

File Edit Format Run Options Window Help

```
import sqlite3
import pandas as pd

conn = sqlite3.connect("college.db")
cursor = conn.cursor()

cursor.execute("""
CREATE TABLE IF NOT EXISTS Student (
    Student_ID INTEGER PRIMARY KEY,
    Name TEXT NOT NULL,
    Department TEXT NOT NULL,
    Age INTEGER,
    City TEXT
)
""")

df = pd.read_csv("C:/Users/hp/Downloads/student/student.csv")

df.to_sql("Student", conn, if_exists="replace", index=False)

print("\n========== STUDENT RECORDS ==========\n")

cursor.execute("SELECT * FROM Student")
records = cursor.fetchall()

for record in records:
    print(record)

cursor.execute("SELECT COUNT(*) FROM Student")
count = cursor.fetchone()[0]

print("\n-------------------------------------")
print("Total Students :", count)

conn.commit()
conn.close()

print("\nData imported successfully into SQLite database.")
```

OUTPUT:

IDLE Shell 3.13.9

File Edit Shell Debug Options Window Help

```
Python 3.13.9 (tags/v3.13.9:8183fa5, Oct 14 2025, 14:09:13) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
================ RESTART: C:/Users/hp/Downloads/student/app.py ================

========== STUDENT RECORDS ==========

(101, 'Aarav', 'CSE', 20, 'Chennai')
(102, 'Diya', 'ECE', 19, 'Bangalore')
(103, 'Rohan', 'IT', 21, 'Hyderabad')
(104, 'Priya', 'EEE', 20, 'Coimbatore')
(105, 'Karthik', 'Civil', 22, 'Madurai')
(106, 'Sneha', 'CSE', 20, 'Salem')
(107, 'Arjun', 'ECE', 21, 'Erode')
(108, 'Meera', 'IT', 19, 'Trichy')
(109, 'Varun', 'CSE', 20, 'Vellore')
(110, 'Ananya', 'EEE', 21, 'Tirunelveli')
(111, 'Rahul', 'Civil', 22, 'Chennai')
(112, 'Harini', 'CSE', 20, 'Bangalore')
(113, 'Vignesh', 'ECE', 19, 'Hyderabad')
(114, 'Divya', 'IT', 20, 'Coimbatore')
(115, 'Surya', 'CSE', 21, 'Madurai')
(116, 'Nisha', 'EEE', 20, 'Salem')
(117, 'Akash', 'Civil', 22, 'Erode')
(118, 'Kavya', 'CSE', 19, 'Trichy')
(119, 'Gokul', 'ECE', 20, 'Vellore')
(120, 'Lakshmi', 'IT', 21, 'Tirunelveli')
(121, 'Reshma', 'CSE', 20, 'Chennai')
(122, 'Manoj', 'EEE', 19, 'Bangalore')
(123, 'Pooja', 'Civil', 21, 'Hyderabad')
(124, 'Keerthana', 'CSE', 20, 'Coimbatore')
(125, 'Aditya', 'ECE', 22, 'Madurai')
(126, 'Ishita', 'IT', 19, 'Salem')
(127, 'Sanjay', 'CSE', 20, 'Erode')
(128, 'Aishwarya', 'EEE', 21, 'Trichy')
(129, 'Naveen', 'Civil', 20, 'Vellore')
(130, 'Ritika', 'CSE', 19, 'Tirunelveli')
(131, 'Sathish', 'ECE', 20, 'Chennai')
(132, 'Monika', 'IT', 21, 'Bangalore')
(133, 'Kiran', 'CSE', 20, 'Hyderabad')
(134, 'Deepika', 'EEE', 22, 'Coimbatore')
```

```
IDLE Shell 3.13.9
File  Edit  Shell  Debug  Options  Window  Help
(116, 'Nisha', 'EEE', 20, 'Salem')
(117, 'Akash', 'Civil', 22, 'Erode')
(118, 'Kavya', 'CSE', 19, 'Trichy')
(119, 'Gokul', 'ECE', 20, 'Vellore')
(120, 'Lakshmi', 'IT', 21, 'Tirunelveli')
(121, 'Reshma', 'CSE', 20, 'Chennai')
(122, 'Manoj', 'EEE', 19, 'Bangalore')
(123, 'Pooja', 'Civil', 21, 'Hyderabad')
(124, 'Keerthana', 'CSE', 20, 'Coimbatore')
(125, 'Aditya', 'ECE', 22, 'Madurai')
(126, 'Ishita', 'IT', 19, 'Salem')
(127, 'Sanjay', 'CSE', 20, 'Erode')
(128, 'Aishwarya', 'EEE', 21, 'Trichy')
(129, 'Naveen', 'Civil', 20, 'Vellore')
(130, 'Ritika', 'CSE', 19, 'Tirunelveli')
(131, 'Sathish', 'ECE', 20, 'Chennai')
(132, 'Monika', 'IT', 21, 'Bangalore')
(133, 'Kiran', 'CSE', 20, 'Hyderabad')
(134, 'Deepika', 'EEE', 22, 'Coimbatore')
(135, 'Praveen', 'Civil', 21, 'Madurai')
(136, 'Shalini', 'CSE', 19, 'Salem')
(137, 'Mohan', 'ECE', 20, 'Erode')
(138, 'Janani', 'IT', 21, 'Trichy')
(139, 'Yash', 'CSE', 20, 'Vellore')
(140, 'Swetha', 'EEE', 19, 'Tirunelveli')
(141, 'Bharath', 'Civil', 22, 'Chennai')
(142, 'Nandhini', 'CSE', 20, 'Bangalore')
(143, 'Ajay', 'ECE', 21, 'Hyderabad')
(144, 'Preethi', 'IT', 20, 'Coimbatore')
(145, 'Hari', 'CSE', 19, 'Madurai')
(146, 'Revathi', 'EEE', 21, 'Salem')
(147, 'Sriram', 'Civil', 22, 'Erode')
(148, 'Gayathri', 'CSE', 20, 'Trichy')
(149, 'Abhinav', 'ECE', 19, 'Vellore')
(150, 'Anu', 'IT', 21, 'Tirunelveli')

--------------------------------------
Total Students : 50

Data imported successfully into SQLite database.
>>>
```

2. Library Management Database (10 Marks)

A library plans to maintain information about its books using an SQLite database. Design a **Book** table containing suitable attributes such as Book ID, Title, Author, Publisher, and Price. Write a Python program to establish a connection with the database, create the table, insert sample records, update the price of a selected book, and retrieve the updated information.

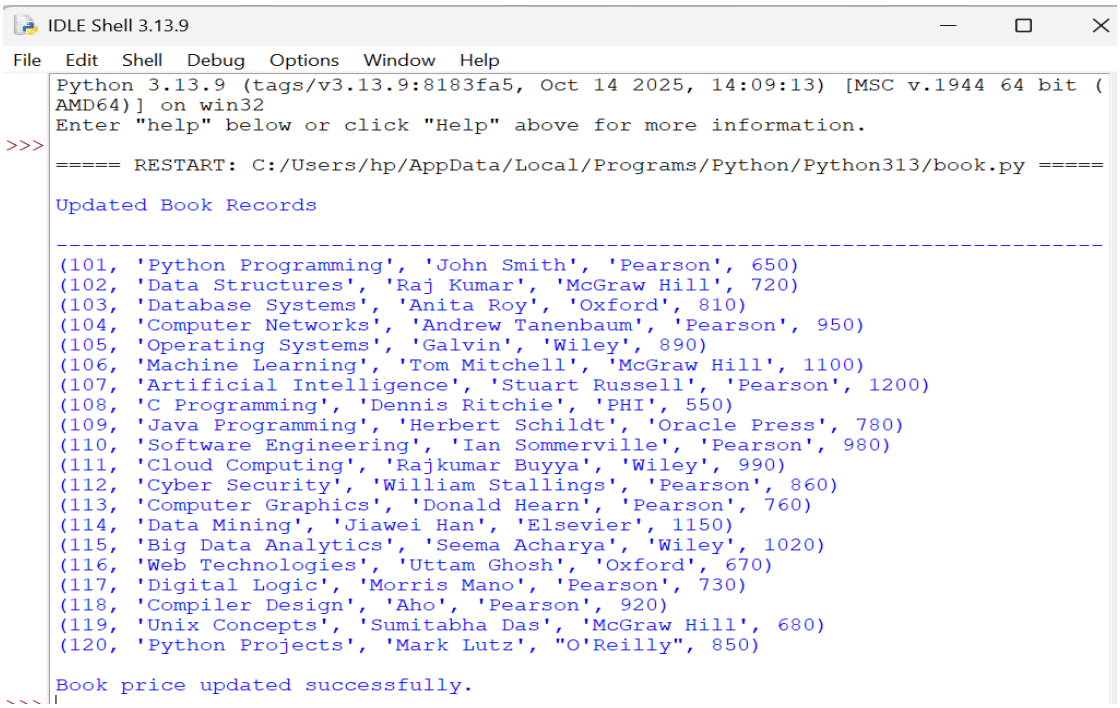
CODE:

book.py - C:/Users/hp/AppData/Local/Programs/Python/Python313/book.py (3.13.9)

File Edit Format Run Options Window Help

```python
import sqlite3
import pandas as pd

conn = sqlite3.connect("library.db")
cursor = conn.cursor()

cursor.execute("""
CREATE TABLE IF NOT EXISTS Book(
    Book_ID INTEGER PRIMARY KEY,
    Title TEXT,
    Author TEXT,
    Publisher TEXT,
    Price REAL
)
""")

df = pd.read_csv("C:/Users/hp/Downloads/BOOK.csv")

df.to_sql("Book", conn, if_exists="replace", index=False)

cursor.execute("""
UPDATE Book
SET Price = 1100
WHERE Book_ID = 106
""")

conn.commit()

cursor.execute("SELECT * FROM Book")

records = cursor.fetchall()

print("\nUpdated Book Records\n")
print("-"*80)

for row in records:
    print(row)

conn.close()
```

OUTPUT:



3. Employee and Department Database Design (10 Marks)

A company wants to store employee information and department details in an SQLite database. Design two related tables, **Department** and **Employee**, by identifying appropriate primary and foreign keys. Write a Python program to connect to the database, create both tables, insert sample data into each table, and display the employee name along with the corresponding department name using an SQL JOIN query.

CODE:

OUTPUT:

```
IDLE Shell 3.13.9
File  Edit  Shell  Debug  Options  Window  Help
Python 3.13.9 (tags/v3.13.9:8183fa5, Oct 14 2025, 14:09:13) [MSC v.1944 64
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:/Users/hp/AppData/Local/Programs/Python/Python313/emplyeeee

Employee Details with Department

--------------------------------------------------------------------------------
(101, 'Aarav', 'Manager', 65000, 'Human Resources')
(102, 'Diya', 'Accountant', 55000, 'Finance')
(103, 'Rohan', 'Marketing Executive', 48000, 'Marketing')
(104, 'Priya', 'Software Engineer', 70000, 'Information Technology')
(105, 'Karthik', 'Sales Executive', 50000, 'Sales')
(106, 'Sneha', 'HR Executive', 45000, 'Human Resources')
(107, 'Arjun', 'Software Developer', 68000, 'Information Technology')
(108, 'Meera', 'Financial Analyst', 60000, 'Finance')
(109, 'Varun', 'System Analyst', 72000, 'Information Technology')
(110, 'Ananya', 'QA Engineer', 58000, 'Quality Assurance')
(111, 'Rahul', 'Support Engineer', 47000, 'Customer Support')
(112, 'Harini', 'Research Associate', 62000, 'Research and Development')
(113, 'Vignesh', 'Admin Officer', 43000, 'Administration')
(114, 'Divya', 'Procurement Officer', 51000, 'Procurement')
(115, 'Surya', 'Sales Manager', 75000, 'Sales')
(116, 'Nisha', 'HR Manager', 70000, 'Human Resources')
(117, 'Akash', 'Software Tester', 54000, 'Quality Assurance')
(118, 'Kavya', 'Customer Executive', 42000, 'Customer Support')
(119, 'Gokul', 'Research Scientist', 85000, 'Research and Development')
(120, 'Lakshmi', 'Finance Manager', 80000, 'Finance')
(121, 'Reshma', 'Software Engineer', 69000, 'Information Technology')
(122, 'Manoj', 'Sales Executive', 52000, 'Sales')
(123, 'Pooja', 'Account Executive', 56000, 'Finance')
(124, 'Keerthana', 'Marketing Manager', 76000, 'Marketing')
(125, 'Aditya', 'System Engineer', 64000, 'Information Technology')
(126, 'Ishita', 'Customer Support', 41000, 'Customer Support')
(127, 'Sanjay', 'QA Analyst', 57000, 'Quality Assurance')
(128, 'Aishwarya', 'Research Intern', 35000, 'Research and Development')
(129, 'Naveen', 'HR Assistant', 39000, 'Human Resources')
(130, 'Ritika', 'Admin Assistant', 38000, 'Administration')
(131, 'Sathish', 'Software Architect', 95000, 'Information Technology')
(132, 'Monika', 'Finance Executive', 53000, 'Finance')
(133, 'Kiran', 'Sales Officer', 47000, 'Sales')
```

4. Hospital Patient Database (10 Marks)

A hospital requires a database to manage patient information. Design an SQLite database with a **Patient** table containing suitable fields and constraints. Write a Python program to connect to the database, create the table, insert patient records, update the contact number of a specified patient, delete a discharged patient's record, and display the remaining records.

CODE:

```
patent.py - C:/Users/hp/AppData/Local/Programs/Python/Python313/patent.py (3.13.9)
File  Edit  Format  Run  Options  Window  Help
import sqlite3
import pandas as pd

conn = sqlite3.connect("hospital.db")

df = pd.read_csv("C:/Users/hp/Downloads/PATENT.csv")

df.to_sql("Patient", conn, if_exists="replace", index=False)

cursor = conn.cursor()

cursor.execute("""
UPDATE Patient
SET Contact_Number='9999999999'
WHERE Patient_ID=105
""")

cursor.execute("""
DELETE FROM Patient
WHERE Patient_ID=103
""")

conn.commit()

cursor.execute("SELECT * FROM Patient")

records = cursor.fetchall()

print("\nRemaining Patient Records\n")
print("-"*100)

for row in records:
    print(row)

conn.close()

print("\nDatabase Updated Successfully.")
```

OUTPUT:

5. Online Course Registration System (10 Marks)

An online learning platform needs to maintain information about students and the courses they have registered for. Design two related SQLite tables, **Student** and **Course_Registration**, using appropriate primary and foreign keys. Write a Python program to connect to the database, create the tables, insert sample records, and retrieve the names of students along with the courses they have registered for using an SQL JOIN operation.

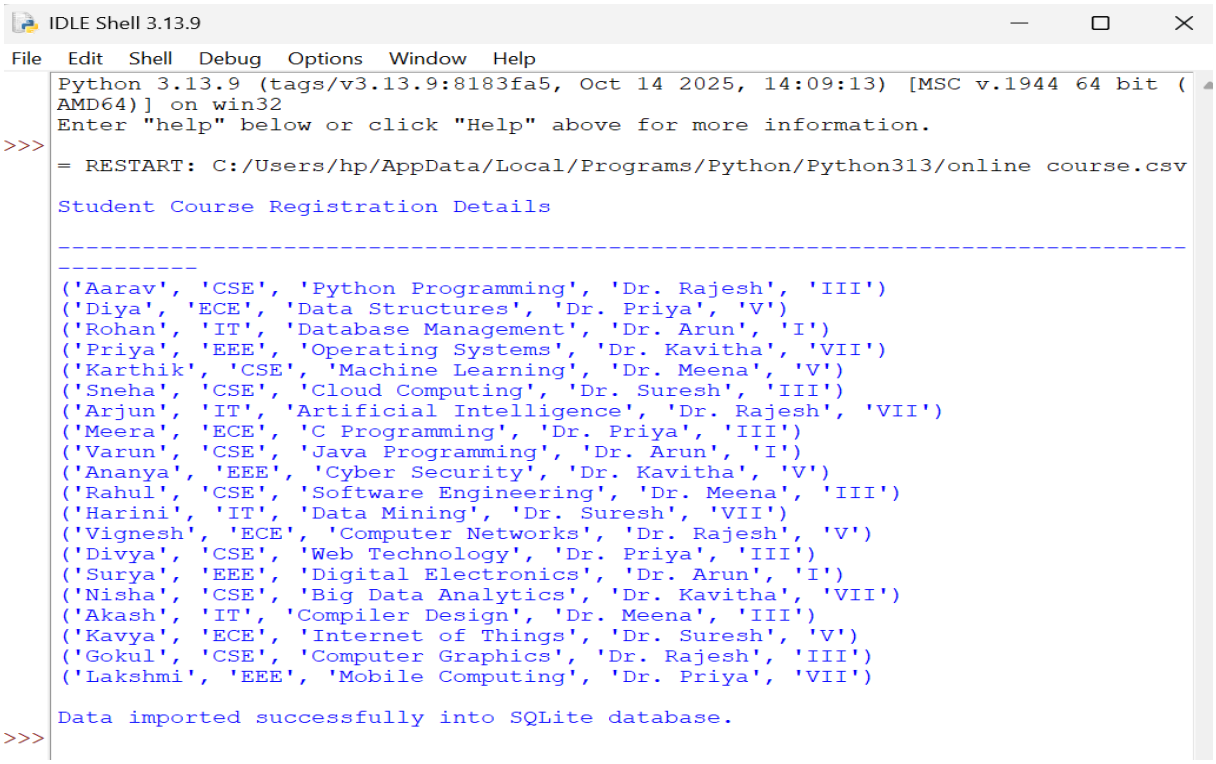
CODE:

online course.csv - C:/Users/hp/AppData/Local/Programs/Python/Python313/online course.csv (3.13.9)

File Edit Format Run Options Window Help

```
import sqlite3
import pandas as pd

conn = sqlite3.connect("course_registration.db")

student_df = pd.read_csv("C:/Users/hp/Downloads/STUDENT.csv")
course_df = pd.read_csv("C:/Users/hp/Downloads/course.csv")

student_df.to_sql("Student", conn, if_exists="replace", index=False)
course_df.to_sql("Course_Registration", conn, if_exists="replace", index=False)

cursor = conn.cursor()

cursor.execute("""
SELECT Student.Student_Name,
       Student.Department,
       Course_Registration.Course_Name,
       Course_Registration.Faculty,
       Course_Registration.Semester
FROM Student
INNER JOIN Course_Registration
ON Student.Student_ID = Course_Registration.Student_ID
""")

records = cursor.fetchall()

print("\nStudent Course Registration Details\n")
print("-"*90)

for row in records:
    print(row)

conn.close()

print("\nData imported successfully into SQLite database.")
```

OUTPUT:


IDLE Shell 3.13.9
File Edit Shell Debug Options Window Help
Python 3.13.9 (tags/v3.13.9:8183fa5, Oct 14 2025, 14:09:13) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/hp/AppData/Local/Programs/Python/Python313/online course.csv
Student Course Registration Details
('Aarav', 'CSE', 'Python Programming', 'Dr. Rajesh', 'III')
('Diya', 'ECE', 'Data Structures', 'Dr. Priya', 'V')
('Rohan', 'IT', 'Database Management', 'Dr. Arun', 'I')
('Priya', 'EEE', 'Operating Systems', 'Dr. Kavitha', 'VII')
('Karthik', 'CSE', 'Machine Learning', 'Dr. Meena', 'V')
('Sneha', 'CSE', 'Cloud Computing', 'Dr. Suresh', 'III')
('Arjun', 'IT', 'Artificial Intelligence', 'Dr. Rajesh', 'VII')
('Meera', 'ECE', 'C Programming', 'Dr. Priya', 'III')
('Varun', 'CSE', 'Java Programming', 'Dr. Arun', 'I')
('Ananya', 'EEE', 'Cyber Security', 'Dr. Kavitha', 'V')
('Rahul', 'CSE', 'Software Engineering', 'Dr. Meena', 'III')
('Harini', 'IT', 'Data Mining', 'Dr. Suresh', 'VII')
('Vignesh', 'ECE', 'Computer Networks', 'Dr. Rajesh', 'V')
('Divya', 'CSE', 'Web Technology', 'Dr. Priya', 'III')
('Surya', 'EEE', 'Digital Electronics', 'Dr. Arun', 'I')
('Nisha', 'CSE', 'Big Data Analytics', 'Dr. Kavitha', 'VII')
('Akash', 'IT', 'Compiler Design', 'Dr. Meena', 'III')
('Kavya', 'ECE', 'Internet of Things', 'Dr. Suresh', 'V')
('Gokul', 'CSE', 'Computer Graphics', 'Dr. Rajesh', 'III')
('Lakshmi', 'EEE', 'Mobile Computing', 'Dr. Priya', 'VII')
Data imported successfully into SQLite database.
>>>

6. Banking Account Management System (10 Marks)

A bank wants to maintain customer account information using an SQLite database. Design an appropriate database containing an **Account** table with suitable attributes and constraints. Write a Python program to connect to the database, create the table, insert customer account details, update the account balance of a specified customer, and display all customer records.

CODE:

account.py - C:/Users/hp/AppData/Local/Programs/Python/Python313/account.py (3.13.9)

File Edit Format Run Options Window Help

```
import sqlite3
import pandas as pd

conn = sqlite3.connect("bank.db")

df = pd.read_csv("C:/Users/hp/Downloads/ACCOUNT.csv")

df.to_sql("Account", conn, if_exists="replace", index=False)

cursor = conn.cursor()

cursor.execute("""
UPDATE Account
SET Balance = 60000
WHERE Account_No = 100005
""")

conn.commit()

cursor.execute("SELECT * FROM Account")

records = cursor.fetchall()

print("\nCustomer Account Records\n")
print("-" * 90)

for row in records:
    print(row)

conn.close()

print("\nAccount balance updated successfully.")
```

OUTPUT:

IDLE Shell 3.13.9

File Edit Shell Debug Options Window Help

```
Python 3.13.9 (tags/v3.13.9:8183fa5, Oct 14 2025, 14:09:13) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
==== RESTART: C:/Users/hp/AppData/Local/Programs/Python/Python313/account.py ===

Customer Account Records

------------------------------------------------------------------------------
----------
(100001, 'Aarav Kumar', 'Savings', 25000, 'Chennai', 9876543210)
(100002, 'Diya Sharma', 'Current', 50000, 'Bangalore', 9876543211)
(100003, 'Rohan Singh', 'Savings', 18000, 'Hyderabad', 9876543212)
(100004, 'Priya Nair', 'Current', 75000, 'Coimbatore', 9876543213)
(100005, 'Karthik Raj', 'Savings', 60000, 'Madurai', 9876543214)
(100006, 'Sneha Iyer', 'Savings', 45000, 'Salem', 9876543215)
(100007, 'Arjun Das', 'Current', 90000, 'Erode', 9876543216)
(100008, 'Meera Joseph', 'Savings', 28000, 'Trichy', 9876543217)
(100009, 'Varun Kumar', 'Current', 65000, 'Vellore', 9876543218)
(100010, 'Ananya Rao', 'Savings', 38000, 'Tirunelveli', 9876543219)
(100011, 'Rahul Verma', 'Current', 55000, 'Chennai', 9876543220)
(100012, 'Harini S', 'Savings', 27000, 'Bangalore', 9876543221)
(100013, 'Vignesh M', 'Current', 82000, 'Hyderabad', 9876543222)
(100014, 'Divya K', 'Savings', 36000, 'Coimbatore', 9876543223)
(100015, 'Surya P', 'Current', 95000, 'Madurai', 9876543224)
(100016, 'Nisha R', 'Savings', 42000, 'Salem', 9876543225)
(100017, 'Akash T', 'Current', 61000, 'Erode', 9876543226)
(100018, 'Kavya L', 'Savings', 30000, 'Trichy', 9876543227)
(100019, 'Gokul V', 'Current', 73000, 'Vellore', 9876543228)
(100020, 'Lakshmi R', 'Savings', 41000, 'Tirunelveli', 9876543229)

Account balance updated successfully.
>>>
```

7. Inventory Management System (10 Marks)

A retail store needs to maintain product inventory using SQLite. Design a **Product** table containing Product ID, Product Name, Category, Quantity, and Unit Price. Write a Python program to create the database, insert sample products, identify products with quantity less than 10, update their stock quantity, and display the updated inventory.

CODE:

product.py - C:/Users/hp/AppData/Local/Programs/Python/Python313/product.py (3.13.9)

File Edit Format Run Options Window Help

```
import sqlite3
import pandas as pd

conn = sqlite3.connect("inventory.db")

df = pd.read_csv("C:/Users/hp/Downloads/PRODUCT.csv")

df.to_sql("Product", conn, if_exists="replace", index=False)

cursor = conn.cursor()

print("\nProducts with Quantity Less Than 10\n")
print("-" * 60)

cursor.execute("""
SELECT * FROM Product
WHERE Quantity < 10
""")

low_stock = cursor.fetchall()

for row in low_stock:
    print(row)

cursor.execute("""
UPDATE Product
SET Quantity = Quantity + 20
WHERE Quantity < 10
""")

conn.commit()

print("\nUpdated Inventory\n")
print("-" * 60)

cursor.execute("SELECT * FROM Product")

records = cursor.fetchall()

for row in records:
    print(row)
```

OUTPUT:

IDLE Shell 3.13.9

File Edit Shell Debug Options Window Help

```
Products with Quantity Less Than 10

------------------------------------------------------------
(101, 'Laptop', 'Electronics', 8, 55000)
(103, 'Keyboard', 'Electronics', 6, 1200)
(105, 'Printer', 'Electronics', 5, 15000)
(107, 'Chair', 'Furniture', 9, 3500)
(111, 'Eraser', 'Stationery', 7, 15)
(113, 'Calculator', 'Electronics', 4, 900)
(116, 'USB Drive', 'Electronics', 3, 700)
(118, 'Headphones', 'Electronics', 6, 2500)
(119, 'Webcam', 'Electronics', 9, 3000)
(120, 'Projector', 'Electronics', 2, 45000)

Updated Inventory

------------------------------------------------------------
(101, 'Laptop', 'Electronics', 28, 55000)
(102, 'Mouse', 'Electronics', 25, 600)
(103, 'Keyboard', 'Electronics', 26, 1200)
(104, 'Monitor', 'Electronics', 12, 9500)
(105, 'Printer', 'Electronics', 25, 15000)
(106, 'Desk', 'Furniture', 15, 7000)
(107, 'Chair', 'Furniture', 29, 3500)
(108, 'Notebook', 'Stationery', 40, 80)
(109, 'Pen', 'Stationery', 100, 20)
(110, 'Pencil', 'Stationery', 80, 10)
(111, 'Eraser', 'Stationery', 27, 15)
(112, 'Marker', 'Stationery', 18, 50)
(113, 'Calculator', 'Electronics', 24, 900)
(114, 'Water Bottle', 'Accessories', 22, 350)
(115, 'School Bag', 'Bags', 10, 1200)
(116, 'USB Drive', 'Electronics', 23, 700)
(117, 'Hard Disk', 'Electronics', 11, 4500)
(118, 'Headphones', 'Electronics', 26, 2500)
(119, 'Webcam', 'Electronics', 29, 3000)
(120, 'Projector', 'Electronics', 22, 45000)

Inventory updated successfully.
>>>
```

8. Course and Faculty Management System (10 Marks)

A university wants to maintain details of faculty members and the courses they teach. Design two related tables, **Faculty** and **Course**, using appropriate primary and foreign keys. Write a Python program to create the database, insert sample records, and display each faculty member along with the courses assigned using an SQL JOIN query.

CODE:

university.py - C:/Users/hp/AppData/Local/Programs/Python/Python313/university.py (3.13.9)

File Edit Format Run Options Window Help

```
import sqlite3
import pandas as pd

conn = sqlite3.connect("university.db")

faculty_df = pd.read_csv("C:/Users/hp/Downloads/FACULTYYY.csv")
course_df = pd.read_csv("C:/Users/hp/Downloads/courseee.csv")

faculty_df.to_sql("Faculty", conn, if_exists="replace", index=False)
course_df.to_sql("Course", conn, if_exists="replace", index=False)

cursor = conn.cursor()

cursor.execute("""
SELECT Faculty.Faculty_Name,
       Faculty.Department,
       Faculty.Designation,
       Course.Course_Name,
       Course.Semester
FROM Faculty
INNER JOIN Course
ON Faculty.Faculty_ID = Course.Faculty_ID
""")

records = cursor.fetchall()

print("\nFaculty and Assigned Courses\n")
print("-" * 90)

for row in records:
    print(row)

conn.close()

print("\nData imported successfully into SQLite database.")
```

OUTPUT:

IDLE Shell 3.13.9

File Edit Shell Debug Options Window Help

```
Python 3.13.9 (tags/v3.13.9:8183fa5, Oct 14 2025, 14:09:13) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
== RESTART: C:/Users/hp/AppData/Local/Programs/Python/Python313/university.py ==

Faculty and Assigned Courses

------------------------------------------------------------------------------
----------
('Dr. Rajesh', 'CSE', 'Professor', 'Artificial Intelligence', 'VII')
('Dr. Rajesh', 'CSE', 'Professor', 'Internet of Things', 'VII')
('Dr. Rajesh', 'CSE', 'Professor', 'Python Programming', 'III')
('Dr. Arun', 'IT', 'Professor', 'Computer Graphics', 'IV')
('Dr. Arun', 'IT', 'Professor', 'Computer Networks', 'V')
('Dr. Arun', 'IT', 'Professor', 'Database Management', 'IV')
('Dr. Arun', 'IT', 'Professor', 'Software Engineering', 'V')
('Dr. Kavitha', 'EEE', 'Assistant Professor', 'Digital Electronics', 'III')
('Dr. Meena', 'CSE', 'Professor', 'Data Mining', 'VI')
('Dr. Meena', 'CSE', 'Professor', 'Data Structures', 'III')
('Dr. Meena', 'CSE', 'Professor', 'Machine Learning', 'VI')
('Dr. Suresh', 'IT', 'Associate Professor', 'Cloud Computing', 'VI')
('Dr. Suresh', 'IT', 'Associate Professor', 'Compiler Design', 'VI')
('Dr. Suresh', 'IT', 'Associate Professor', 'Mobile Computing', 'VI')
('Dr. Divya', 'ECE', 'Assistant Professor', 'C Programming', 'I')
('Dr. Divya', 'ECE', 'Assistant Professor', 'Web Technology', 'V')
('Dr. Karthik', 'CSE', 'Professor', 'Cyber Security', 'VII')
('Dr. Karthik', 'CSE', 'Professor', 'Java Programming', 'IV')
('Dr. Karthik', 'CSE', 'Professor', 'Operating Systems', 'V')
('Dr. Lakshmi', 'EEE', 'Associate Professor', 'Microprocessors', 'IV')

Data imported successfully into SQLite database.
>>>
```

9. Vehicle Registration Database (10 Marks)

A transport department plans to maintain vehicle registration details in an SQLite database. Design a **Vehicle** table with suitable attributes and constraints. Write a Python program to connect to the database, create the table, insert vehicle records, search for a vehicle using its registration number, and display the retrieved information.

CODE:

vehiclle.py - C:/Users/hp/AppData/Local/Programs/Python/Python313/vehiclle.py (3.13.9)

File Edit Format Run Options Window Help

```
import sqlite3
import pandas as pd

conn = sqlite3.connect("vehicle.db")

df = pd.read_csv("C:/Users/hp/Downloads/VEHICLE.csv")

df.to_sql("Vehicle", conn, if_exists="replace", index=False)

cursor = conn.cursor()

reg_no = input("Enter Vehicle Registration Number: ")

cursor.execute("""
SELECT * FROM Vehicle
WHERE Registration_No = ?
""", (reg_no,))

record = cursor.fetchone()

print("\nVehicle Details")
print("-" * 70)

if record:
    print(record)
else:
    print("Vehicle not found.")

conn.close()
```

OUTPUT:

IDLE Shell 3.13.9

File Edit Shell Debug Options Window Help

```
Python 3.13.9 (tags/v3.13.9:8183fa5, Oct 14 2025, 14:09:13) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:/Users/hp/AppData/Local/Programs/Python/Python313/vehiclle.py ===
Enter Vehicle Registration Number: TN05IJ1005

Vehicle Details
----------------------------------------------------------------------
('TN05IJ1005', 'Karthik Raj', 'Car', 'Tata', 'Nexon', 'Grey', 2024)
>>>
```

10. Hospital Appointment Management System (10 Marks)

A hospital maintains separate tables for **Doctors** and **Appointments**. Design both tables using appropriate primary and foreign keys. Write a Python program to create the tables, insert sample records, and display the doctor name along with the appointment details using a JOIN query.

CODE:

doctor.py - C:/Users/hp/AppData/Local/Programs/Python/Python313/doctor.py (3.13.9)

File Edit Format Run Options Window Help

```
import sqlite3
import pandas as pd

conn = sqlite3.connect("hospital.db")

doctor_df = pd.read_csv("C:/Users/hp/Downloads/DOCTOR.csv")
appointment_df = pd.read_csv("C:/Users/hp/Downloads/appointment.csv")

doctor_df.to_sql("Doctors", conn, if_exists="replace", index=False)
appointment_df.to_sql("Appointments", conn, if_exists="replace", index=False)

cursor = conn.cursor()

cursor.execute("""
SELECT Doctors.Doctor_Name,
       Doctors.Specialization,
       Appointments.Patient_Name,
       Appointments.Appointment_Date,
       Appointments.Appointment_Time
FROM Doctors
INNER JOIN Appointments
ON Doctors.Doctor_ID = Appointments.Doctor_ID
""")

records = cursor.fetchall()

print("\nDoctor and Appointment Details")
print("-" * 90)

for row in records:
    print(row)

conn.close()

print("\nData imported successfully into SQLite database.")
```

OUTPUT:

IDLE Shell 3.13.9

File Edit Shell Debug Options Window Help

```
Python 3.13.9 (tags/v3.13.9:8183fa5, Oct 14 2025, 14:09:13) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
==== RESTART: C:/Users/hp/AppData/Local/Programs/Python/Python313/doctor.py ====

Doctor and Appointment Details
------------------------------------------------------------------------------
----------
('Dr. Rajesh', 'Cardiologist', 'Aarav Kumar', '2026-08-05', '09:00')
('Dr. Rajesh', 'Cardiologist', 'Rahul Verma', '2026-08-06', '09:00')
('Dr. Priya', 'Dermatologist', 'Diya Sharma', '2026-08-05', '09:30')
('Dr. Priya', 'Dermatologist', 'Harini S', '2026-08-06', '09:30')
('Dr. Arun', 'Orthopedic', 'Rohan Singh', '2026-08-05', '10:00')
('Dr. Arun', 'Orthopedic', 'Vignesh M', '2026-08-06', '10:00')
('Dr. Kavitha', 'Pediatrician', 'Divya K', '2026-08-06', '10:30')
('Dr. Kavitha', 'Pediatrician', 'Priya Nair', '2026-08-05', '10:30')
('Dr. Meena', 'Neurologist', 'Karthik Raj', '2026-08-05', '11:00')
('Dr. Meena', 'Neurologist', 'Surya P', '2026-08-06', '11:00')
('Dr. Suresh', 'ENT Specialist', 'Nisha R', '2026-08-06', '11:30')
('Dr. Suresh', 'ENT Specialist', 'Sneha Iyer', '2026-08-05', '11:30')
('Dr. Divya', 'Gynecologist', 'Akash T', '2026-08-06', '12:00')
('Dr. Divya', 'Gynecologist', 'Arjun Das', '2026-08-05', '12:00')
('Dr. Karthik', 'General Physician', 'Kavya L', '2026-08-06', '12:30')
('Dr. Karthik', 'General Physician', 'Meera Joseph', '2026-08-05', '12:30')
('Dr. Lakshmi', 'Ophthalmologist', 'Gokul V', '2026-08-06', '02:00')
('Dr. Lakshmi', 'Ophthalmologist', 'Varun Kumar', '2026-08-05', '02:00')
('Dr. Nisha', 'Dentist', 'Ananya Rao', '2026-08-05', '02:30')
('Dr. Nisha', 'Dentist', 'Lakshmi R', '2026-08-06', '02:30')

Data imported successfully into SQLite database.
>>>
```